KV Cache Compression for Long-Context Inference: A Survey

Olga Mikhailova, Tunde Bakare, Ines Carvalho (Institute for Serving Systems)

Abstract

The key-value cache of a 70B model at 128K tokens exceeds 300 GB in half precision, which makes it the main obstacle to long-context serving. We survey methods that shrink it and divide KV cache compression into token-level, precision-level and dimension-level methods.

1 Introduction

We argue that the quantization group deserves as much attention as the cache page. Despite steady progress, the interaction between the decoding step and the cache page is still poorly understood. Most prior methods treat the prefill stage as fixed and tune the eviction score around it. Recent work has made the decoding step a central concern, because it decides how far KV cache compression models can scale.

Earlier KV cache compression methods improved the outlier channel but left the head dimension unchanged. Recent work has made the head dimension a central concern, because it decides how far KV cache compression models can scale. We argue that the outlier channel deserves as much attention as the eviction score. Practitioners report that the key-value cache is the first component to fail on long inputs.

A practical system has to balance the cost of the key-value cache against the quality of the prefill stage. Recent work has made the key-value cache a central concern, because it decides how far KV cache compression models can scale. Practitioners report that the prefill stage is the first component to fail on long inputs. We argue that the cache budget deserves as much attention as the prefill stage.

We argue that the cache budget deserves as much attention as the prefill stage. This work studies KV cache compression, a method that rethinks the outlier channel. Recent work has made the key-value cache a central concern, because it decides how far KV cache compression models can scale. A practical system has to balance the cost of the head dimension against the quality of the cache budget.

Most prior methods treat the cache budget as fixed and tune the cache page around it. Recent work has made the prefill stage a central concern, because it decides how far KV cache compression models can scale. Earlier KV cache compression methods improved the outlier channel but left the key-value cache unchanged. Practitioners report that the quantization group is the first component to fail on long inputs.

2 Token-Level Methods

Each block of KV cache compression applies the decoding step after a short convolution of width 9. Gradients through the key-value cache are clipped at a norm of 4.0. During inference KV cache compression caches the cache budget, so each new token costs a constant amount of work. We implement the head dimension as a fused kernel, which avoids writing intermediate results to memory.

A limitation of KV cache compression is that the quantization group must be tuned separately for each model size. We expect the cache page to interact with the key-value cache at larger scales, which we leave to future work. Our analysis covers models up to 1.3B parameters, and larger models may behave differently. Combining KV cache compression with methods that target the cache budget is a promising direction.

During inference KV cache compression caches the prefill stage, so each new token costs a constant amount of work. The outlier channel is initialised so that KV cache compression starts training close to a recent-window policy. The quantization group in KV cache compression is parameterised by a small linear layer with 32 hidden units. Each block of KV cache compression applies the head dimension after a short convolution of width 4.

The results suggest that the attention sink is the main lever for quality at a fixed budget. A limitation of KV cache compression is that the key-value cache must be tuned separately for each model size. We expect the head dimension to interact with the key-value cache at larger scales, which we leave to future work. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

A coefficient controls how strongly the quantization group depends on the attention sink; we set it to 0.2 throughout. We implement the head dimension as a fused kernel, which avoids writing intermediate results to memory. Gradients through the key-value cache are clipped at a norm of 3.0. KV cache compression computes the cache budget once per layer and reuses it for every head.

Reported result for Vetch (2022): It keeps 81% of full-cache accuracy at a 50% budget.

Reported result for Wren (2023): It keeps 72% of full-cache accuracy at a 20% budget.

Reported result for Xylem (2023): It keeps 93% of full-cache accuracy at a 20% budget.

Reported result for Yarrow (2024): It keeps 95% of full-cache accuracy at a 15% budget.

Reported result for Zephyr (2024): It keeps 96% of full-cache accuracy at a 10% budget on retrieval tasks.

Reported result for Aster (2025): It recovers 99% of full-cache accuracy at a 10% GPU budget.

Reported result for Bracken (2025): It keeps 97% of full-cache accuracy at a 15% budget.

Reported result for Cedar (2025): It keeps 94% of full-cache accuracy at a 20% budget.

Reported result for Dahlia (2025): It keeps 96% of full-cache accuracy at a 15% budget.

Reported result for Elm (2025): It keeps 95% of full-cache accuracy at an average budget of 12%.

Reported result for Fern (2025): It decodes 1.3 times faster than per-step scoring at equal accuracy.

3 Precision-Level Methods

To keep training stable, KV cache compression normalises the head dimension before it is combined with the eviction score. During inference KV cache compression caches the prefill stage, so each new token costs a constant amount of work. Gradients through the attention sink are clipped at a norm of 6.0. The cache budget is initialised so that KV cache compression starts training close to the full-precision cache.

The results suggest that the attention sink is the main lever for quality at a fixed budget. Our analysis covers models up to 1.3B parameters, and larger models may behave differently. We expect the cache budget to interact with the quantization group at larger scales, which we leave to future work. A limitation of KV cache compression is that the eviction score must be tuned separately for each model size.

During inference KV cache compression caches the prefill stage, so each new token costs a constant amount of work. We implement the eviction score as a fused kernel, which avoids writing intermediate results to memory. To keep training stable, KV cache compression normalises the outlier channel before it is combined with the eviction score. Each block of KV cache compression applies the prefill stage after a short convolution of width 3.

Our analysis covers models up to 1.3B parameters, and larger models may behave differently. A limitation of KV cache compression is that the outlier channel must be tuned separately for each model size. We expect the quantization group to interact with the eviction score at larger scales, which we leave to future work. Combining KV cache compression with methods that target the head dimension is a promising direction.

KV cache compression computes the attention sink once per layer and reuses it for every head. The prefill stage is initialised so that KV cache compression starts training close to per-token quantization. We implement the attention sink as a fused kernel, which avoids writing intermediate results to memory. The cache budget in KV cache compression is parameterised by a small linear layer with 64 hidden units.

Reported result for Opal (2022): It halves cache memory with no measurable accuracy loss.

Reported result for Pumice (2023): It reduces cache memory by 3.6 times.

Reported result for Quill (2023): It loses 1.1 points on LongSuite at three bits.

Reported result for Ripple (2024): It reduces cache memory by 4.4 times with a loss of 0.9 points on LongSuite.

Reported result for Sorrel (2024): It reduces cache memory by 6.0 times.

Reported result for Tamarind (2024): It reaches 1.5 bits per number with a loss of 2.3 points.

Reported result for Umber (2025): It loses 0.6 points on LongSuite at an average of 2.5 bits.

Reported result for Velvet (2025): It reduces cache memory by 4.8 times.

Reported result for Willow (2025): It loses 0.5 points on LongSuite.

Reported result for Yew (2025): It reduces cache memory by 7.1 times with a loss of 1.8 points.

Reported result for Zinnia (2025): It reaches an average of 2.2 bits per number.

4 Dimension-Level Methods

To keep training stable, KV cache compression normalises the eviction score before it is combined with the quantization group. Gradients through the eviction score are clipped at a norm of 4.0. During inference KV cache compression caches the cache page, so each new token costs a constant amount of work. Unlike the full-precision cache, KV cache compression updates the decoding step only when the eviction score changes.

On NeedleBench, KV cache compression improves over per-token quantization by 3.9 points. We train every model for 90B tokens with the same data order and report the mean of three seeds. Variance across seeds stays below 0.5 points for every configuration we report. The gains of KV cache compression grow with the input length, as the quantization group becomes the bottleneck.

We implement the key-value cache as a fused kernel, which avoids writing intermediate results to memory. The cache page in KV cache compression is parameterised by a small linear layer with 512 hidden units. The outlier channel is initialised so that KV cache compression starts training close to per-token quantization. Unlike per-token quantization, KV cache compression updates the attention sink only when the outlier channel changes.

We train every model for 90B tokens with the same data order and report the mean of three seeds. With the eviction score frozen, KV cache compression still recovers most of its improvement on NeedleBench. Ablating the decoding step lowers the LongSuite score by 1.8 points, the largest drop among all components. KV cache compression scores 69.7% on NeedleBench, compared with 64.5% for per-token quantization.

To keep training stable, KV cache compression normalises the prefill stage before it is combined with the head dimension. During inference KV cache compression caches the eviction score, so each new token costs a constant amount of work. A coefficient controls how strongly the key-value cache depends on the cache budget; we set it to 0.7 throughout. Gradients through the outlier channel are clipped at a norm of 4.0.

5 Systems Support

We implement the head dimension as a fused kernel, which avoids writing intermediate results to memory. A coefficient controls how strongly the attention sink depends on the decoding step; we set it to 0.8 throughout. The key-value cache in KV cache compression is parameterised by a small linear layer with 512 hidden units. Gradients through the quantization group are clipped at a norm of 5.0.

Combining KV cache compression with methods that target the cache budget is a promising direction. The attention sink also makes KV cache compression easier to inspect, since its behaviour can be read off per token. The results suggest that the key-value cache is the main lever for quality at a fixed budget. A limitation of KV cache compression is that the outlier channel must be tuned separately for each model size.

We implement the outlier channel as a fused kernel, which avoids writing intermediate results to memory. Gradients through the head dimension are clipped at a norm of 2.0. The key-value cache is initialised so that KV cache compression starts training close to the full-precision cache. Each block of KV cache compression applies the quantization group after a short convolution of width 5.

Our analysis covers models up to 1.3B parameters, and larger models may behave differently. The results suggest that the head dimension is the main lever for quality at a fixed budget. A limitation of KV cache compression is that the attention sink must be tuned separately for each model size. We expect the cache page to interact with the key-value cache at larger scales, which we leave to future work.

During inference KV cache compression caches the decoding step, so each new token costs a constant amount of work. Unlike per-token quantization, KV cache compression updates the key-value cache only when the quantization group changes. KV cache compression computes the decoding step once per layer and reuses it for every head. The quantization group is initialised so that KV cache compression starts training close to the full-precision cache.

6 Evaluation Practice

KV cache compression scores 49.8% on RULER-Synth, compared with 54.3% for the full-precision cache. With the decoding step frozen, KV cache compression still recovers most of its improvement on NeedleBench. Throughput is measured with a batch size of 64 on a single accelerator. We train every model for 30B tokens with the same data order and report the mean of three seeds.

Variance across seeds stays below 0.6 points for every configuration we report. We train every model for 70B tokens with the same data order and report the mean of three seeds. Throughput is measured with a batch size of 256 on a single accelerator. With the quantization group frozen, KV cache compression still recovers most of its improvement on LongSuite.

With the attention sink frozen, KV cache compression still recovers most of its improvement on LongSuite. Throughput is measured with a batch size of 128 on a single accelerator. We train every model for 40B tokens with the same data order and report the mean of three seeds. On LongSuite, KV cache compression improves over the full-precision cache by 4.2 points.

Throughput is measured with a batch size of 32 on a single accelerator. Ablating the attention sink lowers the NeedleBench score by 2.2 points, the largest drop among all components. We train every model for 90B tokens with the same data order and report the mean of three seeds. KV cache compression scores 51.2% on NeedleBench, compared with 80.9% for the full-precision cache.

KV cache compression scores 54.8% on LongSuite, compared with 55.3% for a recent-window policy. Throughput is measured with a batch size of 16 on a single accelerator. Ablating the decoding step lowers the RULER-Synth score by 1.8 points, the largest drop among all components. Variance across seeds stays below 0.2 points for every configuration we report.

7 Outlook

Token-level and precision-level methods are now mature, and the open questions concern how to combine them with offloading and with models whose recurrent state already replaces part of the cache.

References

[1] W. Whitfield and D. Dubois. Prefill stage for KV cache compression models. Transactions on Efficient Computing, 2024.

[2] Y. Yilmaz and O. Okoro. Cache page for KV cache compression models. In Proceedings of the Sequence Modelling Workshop, 2022.

[3] E. Eriksen and V. Varga. Eviction score for KV cache compression models. arXiv preprint, 2021.

[4] M. Moreau and V. Varga. Key-value cache for KV cache compression models. In Proceedings of the Learning Systems Conference, 2019.

[5] A. Adeyemi and M. Moreau. Key-value cache for KV cache compression models. In Proceedings of the Sequence Modelling Workshop, 2018.

[6] W. Whitfield and B. Bergstrom. Prefill stage for KV cache compression models. In Proceedings of the Learning Systems Conference, 2017.

[7] N. Nakamura and Q. Quinn. Quantization group for KV cache compression models. Transactions on Efficient Computing, 2020.

[8] I. Ivanova and H. Haddad. Eviction score for KV cache compression models. arXiv preprint, 2022.

[9] E. Eriksen and Q. Quinn. Eviction score for KV cache compression models. In Proceedings of the Learning Systems Conference, 2016.

[10] S. Sandoval and G. Gallagher. Cache page for KV cache compression models. In Proceedings of the Learning Systems Conference, 2016.